

MANUAL DE UNITY Y GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para el desarrollo colaborativo de videojuegos en macOS con Apple Silicon (M1/M2/M3/M4), Zsh, Keychain, UnityYAMLMerge y GameCI

[Git LFS & Assets Binarios](#)[UnityYAMLMerge 3-Way Driver](#)[Visible Meta Files & GUIDs](#)[Multi-Scene & Prefab Isolation](#)[GameCI & GitHub Actions](#)[Catálogo de Incidentes Críticos](#)

Manual de Unity y GitHub: De Novato a Avanzado en macOS (Apple Silicon M1/M2/M3/M4)

Plataforma: macOS Sonoma / Sequoia / versiones modernas con arquitectura ARM64 (Apple Silicon)

Motor: Unity 2022 LTS / Unity 6 (6000.x) optimizado para Apple Silicon

Herramientas: Zsh, Git 2.40+, Git LFS, GitHub CLI (`gh`), GitHub Desktop, Apple Keychain, Homebrew en `/opt/homebrew`, Unity Hub, UnityYAMLMerge

Índice de Contenidos

1. **Parte I: Fundamentos y Preparación del Entorno Unity en Apple Silicon**
 - 1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora
 - 1.2 Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Homebrew
 - 1.3 Configuración Crítica del Editor: Visible Meta Files y Force Text
 - 1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs
 - 1.5 El Archivo `.gitignore` Oficial y Optimizado para macOS y Unity
 - 1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`
 - 1.7 Arquitectura de Proyectos AAA: Separación de Assets Propios (`_Project/`) vs Plugins
2. **Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)**
 - 2.1 Inicializar y Publicar un Proyecto en GitHub (Terminal Zsh y GitHub Desktop)
 - 2.2 Clonación Correcta de Proyectos con Git LFS en macOS
 - 2.3 El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits
 - 2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos
 - 2.5 Sincronización sin Romper la Cache (`pull --rebase` vs Fetch en Desktop)
3. **Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)**
 - 3.1 Estrategia de Ramas en Equipos de Videojuegos (Zsh y Desktop)
 - 3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)
 - 3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes
 - 3.4 Configuración de UnityYAMLMerge en macOS y Personalización de Fallback en `mergespecfile.txt`
 - 3.5 Uso de Git Stash, Historial y Reversión Segura (Zsh y Desktop)
 - 3.6 Modularización de Código con Assembly Definitions (`.asmdef` y `.asmref`)
 - 3.7 Unity Accelerator: Aceleración de Descargas y Caché de Importación en LAN
4. **Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente**

- 4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity
 - 4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)
 - 4.3 Tema 3: Conflictos en Scripts C# (`.cs`)
 - 4.4 Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge (Zsh y Desktop)
 - 4.5 Tema 5: Forzar una Versión Completa de Asset (`--ours` vs `--theirs`)
 - 4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull
 - 4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS
 - 4.8 Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock
 - 4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano
5. Parte V: Herramientas Modernas de Productividad Avanzada
- 5.1 Git Worktrees en macOS: Múltiples Ramas sin Recargar `Library/`
 - 5.2 Depuración Binaria con Git Bisect y Blame en C#
 - 5.3 GitHub Codespaces desde la Terminal de Mac
 - 5.4 GitHub Copilot CLI en Zsh
 - 5.5 Git Hooks y Validación Pre-commit de Archivos `.meta` en macOS
 - 5.6 Descargas Parciales y Ahorro de Cuota de Git LFS con `lfs.fetchexclude` y `git sparse-checkout`
6. Parte VI: Gestión de Paquetes UPM y Dependencias
- 6.1 Instalación de Paquetes Mediante URLs de Git en UPM
 - 6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens
 - 6.3 Addressables Asset System frente a `Resources/` : Versionado y Despliegue en CDN
7. Parte VII: Automatización CI/CD con GitHub Actions y GameCI
- 7.1 Arquitectura de GameCI para macOS Standalone (Apple Silicon)
 - 7.2 Activación de Licencias de Unity en GitHub Actions
 - 7.3 Pipeline Automatizado: Tests y Compilación StandaloneOSX (.app / .dmg)
 - 7.4 Subida Automática de Builds a GitHub Releases
 - 7.5 Pruebas Automatizadas con Code Coverage y Reportes en GitHub Actions
8. Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos
- 8.1 Branch Protection Rules y Rulesets
 - 8.2 Gestión de Secretos para APIs de Juegos (Apple Game Center, Steam)
 - 8.3 Gobernanza con `CODEOWNERS` para Artistas y Programadores
9. Parte IX: Distribución y Despliegue con GitHub Releases
- 9.1 Creación Automatizada de Releases con Tags Semánticos
 - 9.2 Empaquetado y Publicación de Archivos DMG y Aplicaciones macOS

10. Parte X: Catálogo Maestro de Incidentes Críticos de Unity en macOS

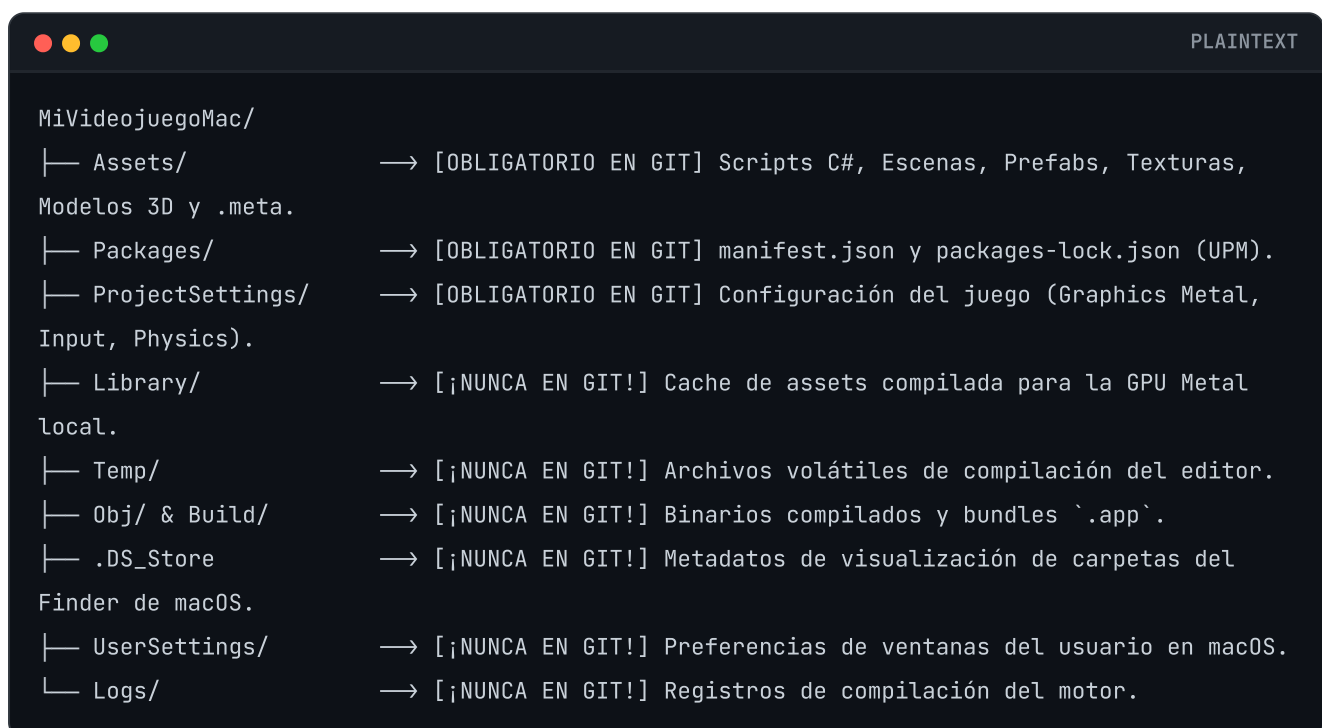
- 10.1 Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs
 - 10.2 Incidente 2: Escena Corrupta por Edición Manual o Conflicto Mal Resuelto
 - 10.3 Incidente 3: Subida Accidental de la Carpeta `Library/` o Archivos `.DS_Store`
 - 10.4 Incidente 4: Repositorio Bloqueado por Superar el Límite de 100 MB
 - 10.5 Incidente 5: Shaders Magenta / Rosados tras Clonar en macOS
 - 10.6 Incidente 6: Límite de Almacenamiento y Ancho de Banda de Git LFS Superado
 - 10.7 Incidente 7: Desfase de Versiones Menores del Editor de Unity
 - 10.8 Incidente 8: Archivos Bloqueados por Procesos de Unity en Ejecución
 - 10.9 Incidente 9: Fuga de Claves Privadas en ScriptableObjects o Certificados Apple
 - 10.10 Incidente 10: Regeneración Limpia y Segura de la Caché Local en macOS
-

Parte I: Fundamentos y Preparación del Entorno

Unity en Apple Silicon

1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora

En macOS con chips Apple Silicon (M1/M2/M3/M4), Unity compila assets a gran velocidad gracias a la memoria unificada. Sin embargo, la estructura interna debe respetarse estrictamente para evitar subir cachés innecesarias a Git:



MiVideojuegoMac/	
├─ Assets/	→ [OBLIGATORIO EN GIT] Scripts C#, Escenas, Prefabs, Texturas, Modelos 3D y .meta.
├─ Packages/	→ [OBLIGATORIO EN GIT] manifest.json y packages-lock.json (UPM).
├─ ProjectSettings/	→ [OBLIGATORIO EN GIT] Configuración del juego (Graphics Metal, Input, Physics).
├─ Library/	→ [¡NUNCA EN GIT!] Cache de assets compilada para la GPU Metal local.
├─ Temp/	→ [¡NUNCA EN GIT!] Archivos volátiles de compilación del editor.
├─ Obj/ & Build/	→ [¡NUNCA EN GIT!] Binarios compilados y bundles `.app`.
├─ .DS_Store	→ [¡NUNCA EN GIT!] Metadatos de visualización de carpetas del Finder de macOS.
├─ UserSettings/	→ [¡NUNCA EN GIT!] Preferencias de ventanas del usuario en macOS.
└─ Logs/	→ [¡NUNCA EN GIT!] Registros de compilación del motor.

[↑ Volver al Índice de Contenidos](#)

1.2 Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Homebrew

En macOS con Apple Silicon, Homebrew se instala en `/opt/homebrew`. Ejecuta en la terminal **Zsh**:

```
# 1. Instalar Git nativo ARM64, Git LFS, GitHub CLI y GitHub Desktop
brew install git git-lfs gh
brew install --cask github

# 2. Inicializar los filtros de Git LFS
git lfs install

# 3. Configurar integración con el Llavero de macOS (Apple Keychain)
git config --global credential.helper osxkeychain
```

¿Qué hace este comando?

Instala las versiones optimizadas para ARM64 de Git, Git LFS y GitHub CLI, instala la aplicación gráfica oficial de **GitHub Desktop** para macOS, y activa `osxkeychain` para que macOS gestione de forma segura y transparente las credenciales de GitHub con Touch ID.

[↑ Volver al Índice de Contenidos](#)

1.3 Configuración Crítica del Editor: Visible Meta Files y Force Text

Antes de inicializar Git, verifica estos parámetros en el Editor de Unity:

1. Abre tu proyecto en Unity en macOS.
2. Navega al menú superior: **Unity** → **Project Settings** → **Version Control**.
 - En **Mode**, selecciona estrictamente: `Visible Meta Files`.
3. Navega a **Unity** → **Project Settings** → **Editor**.
 - En **Asset Serialization Mode**, selecciona estrictamente: `Force Text`.

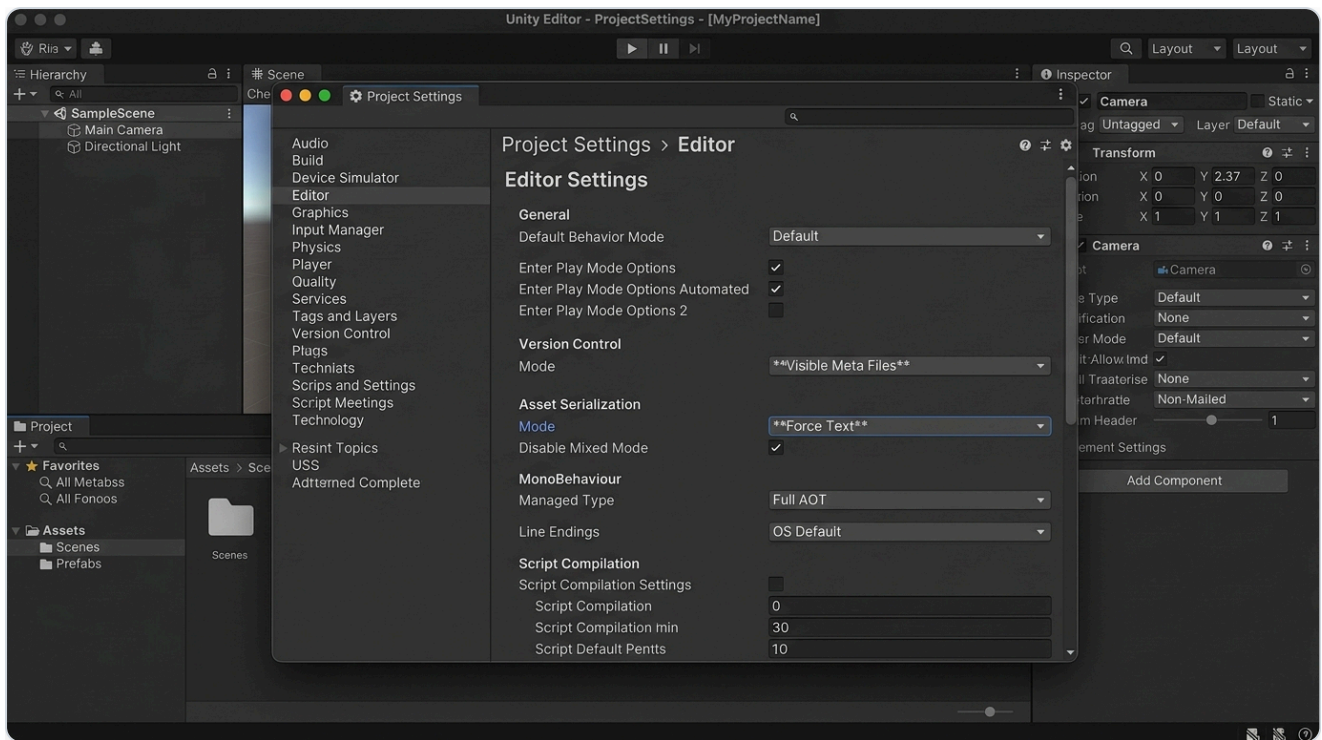


Figura 1.1: Configuración en Unity Project Settings (Editor) en macOS estableciendo Visible Meta Files y Force Text para serialización YAML.

! IMPORTANT

Force Text obliga al motor a serializar escenas (`.unity`), prefabs (`.prefab`) y materiales (`.mat`) en formato de texto plano **YAML**, permitiendo a Git generar diffs legibles y resolver fusiones.

[↑ Volver al Índice de Contenidos](#)

1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs

Cada archivo en la carpeta `Assets/` está emparejado con un `.meta` que contiene su GUID único:

```
YAML
fileFormatVersion: 2
guid: a7c12f80e94b4d6b8890123456789abc
```

CAUTION

LA REGLA FUNDAMENTAL:

Si renombas o mueves un asset desde el Finder de macOS o desde Zsh, **debes renombrar o mover su archivo `.meta` exactamente igual**. Nunca hagas commit de un asset sin su respectivo `.meta` acompañante para evitar la pérdida de referencias ("Missing Script").

[↑ Volver al Índice de Contenidos](#)

1.5 El Archivo `.gitignore` Oficial y Optimizado para macOS y Unity

Crea `.gitignore` en la raíz del repositorio:

```
# =====
# .gitignore para Unity en macOS (Apple Silicon)
# =====

[ Ll ]ibrary/
[ Tt ]emp/
[ 0o ]bj/
[ Bb ]uild/
[ Bb ]uilds/
[ Ll ]ogs/
[ Uu ]ser[ Ss ]ettings/
[ Mm ]emoryCaptures/

# Asset server cache
sysinfo.txt
*.stackdump

# Entornos de desarrollo macOS (Xcode / Rider / VS Code)
.vs/
.idea/
*.csproj
*.unityproj
*.sln
*.suo
*.user
*.userprefs
*.pidb
*.booproj
*.svd
*.pdb
*.opendb
*.VC.db

# Compilaciones de macOS (.app / .dmg) y paquetes
*.app
*.dmg
*.apk
*.unitypackage

# Archivos específicos de macOS
.DS_Store
```

```
.AppleDouble  
.LSOVERRIDE  
._*  
.DocumentRevisions-V100  
.fsevents  
.Spotlight-V100  
.TemporaryItems  
.Trashes
```

[↑ Volver al Índice de Contenidos](#)

1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`

Guarda este archivo `.gitattributes` en la raíz del proyecto:

```
# =====
# Configuración Git LFS para Unity en macOS
# =====

# Formatos 3D y Rigs
*.fbx filter=lfs diff=lfs merge=lfs -text
*.obj filter=lfs diff=lfs merge=lfs -text
*.blend filter=lfs diff=lfs merge=lfs -text
*.dae filter=lfs diff=lfs merge=lfs -text
*.max filter=lfs diff=lfs merge=lfs -text

# Texturas e Imágenes de Alta Resolución
*.psd filter=lfs diff=lfs merge=lfs -text
*.tga filter=lfs diff=lfs merge=lfs -text
*.png filter=lfs diff=lfs merge=lfs -text
*.jpg filter=lfs diff=lfs merge=lfs -text
*.jpeg filter=lfs diff=lfs merge=lfs -text
*.exr filter=lfs diff=lfs merge=lfs -text
*.hdr filter=lfs diff=lfs merge=lfs -text
*.tif filter=lfs diff=lfs merge=lfs -text
*.tiff filter=lfs diff=lfs merge=lfs -text

# Audio y Vídeo
*.wav filter=lfs diff=lfs merge=lfs -text
*.mp3 filter=lfs diff=lfs merge=lfs -text
*.ogg filter=lfs diff=lfs merge=lfs -text
*.mp4 filter=lfs diff=lfs merge=lfs -text
*.mov filter=lfs diff=lfs merge=lfs -text

# Paquetes Comprimidos y Tipografías
*.zip filter=lfs diff=lfs merge=lfs -text
*.7z filter=lfs diff=lfs merge=lfs -text
*.otf filter=lfs diff=lfs merge=lfs -text
*.ttf filter=lfs diff=lfs merge=lfs -text

# Bloqueo Concurrente para Binarios
*.fbx lockable
*.psd lockable
*.blend lockable
*.wav lockable
```

```
# Fusión Semántica de Unity y Finales de Línea LF
*.cs text diff=csharp eol=lf
*.json text eol=lf
*.shader text eol=lf
*.unity merge=unityyamlmerge eol=lf
*.prefab merge=unityyamlmerge eol=lf
*.mat merge=unityyamlmerge eol=lf
*.asset merge=unityyamlmerge eol=lf
*.meta text eol=lf
```

[↑ Volver al Índice de Contenidos](#)

1.7 Arquitectura de Proyectos AAA: Separación de Assets Propios (`\_Project/`) vs Plugins

En macOS, mantener una jerarquía limpia en la raíz de `Assets/` es crucial cuando colaboran múltiples departamentos. Si instalas paquetes desde la Unity Asset Store directamente en `Assets/`, sus carpetas se entremezclan con tus scripts y escenas nativos, generando confusión en las revisiones de código en GitHub.

Patrón Recomendado: La Carpeta Raíz `\_Project/`

Crea una carpeta de proyecto con prefijo de guion bajo para ubicarla en la cima del Project Window de Unity en macOS:

```
Assets/
├── _Project/           → [CÓDIGO FUENTE Y ASSETS DEL ESTUDIO]
│   ├── Art/           → Modelos 3D, Texturas Metal/PBR y Materiales
│   ├── Audio/         → SFX en WAV y Pistas Musicales
│   ├── Core/          → Arquitectura, Controladores y Singletons C#
│   ├── Gameplay/      → Mecánicas jugables, Físicas e Inputs
│   ├── Prefabs/       → Prefabs y Variantes Modulares
│   ├── Scenes/        → Escenas Maestras y Aditivas
│   └── UI/            → Canvas, HUD y Assets TextMeshPro
├── Plugins/           → Frameworks nativos de Apple (.bundle / .dylib de macOS/iOS)
└── ThirdParty/        → Assets importados de la Asset Store (solo lectura)
```

 TIP

Al confinar el desarrollo activo a `Assets/_Project/`, facilitas las revisiones de Pull Requests filtrando únicamente esa ruta y evitas sobrescribir configuraciones críticas al actualizar librerías de terceros en `ThirdParty/`.

[↑ Volver al Índice de Contenidos](#)

Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)

2.1 Inicializar y Publicar un Proyecto en GitHub (Terminal Zsh y GitHub Desktop)

Modalidad A: Vía Terminal Zsh



```
cd ~/UnityProjects/MiVideojuego
git init
git lfs install
git add .gitattributes .gitignore
git commit -m "chore: inicializar configuracion Unity con LFS y gitignore"
git branch -M main
gh repo create MiVideojuegoMac --private --source=. --remote=origin --push
```

¿Qué hace este comando?

Inicializa el repositorio local, activa Git LFS, crea el commit fundacional y publica el repositorio como privado en GitHub vinculando la rama `main`.

Modalidad B: Vía GitHub Desktop en macOS

1. Abre **GitHub Desktop** desde el Launchpad o Spotlight (`Cmd + Espacio`).
2. Pulsa en **File** → **Add Local Repository** (o `Cmd + O`).
3. Selecciona la carpeta de tu juego en tu Mac.
4. Si la carpeta aún no tiene repositorio, pulsa en el enlace azul **"create a repository"**.
5. Pulsa en el botón azul **Add Repository**.
6. En la barra superior, pulsa en **Publish repository** para crearlo en tu cuenta de GitHub.

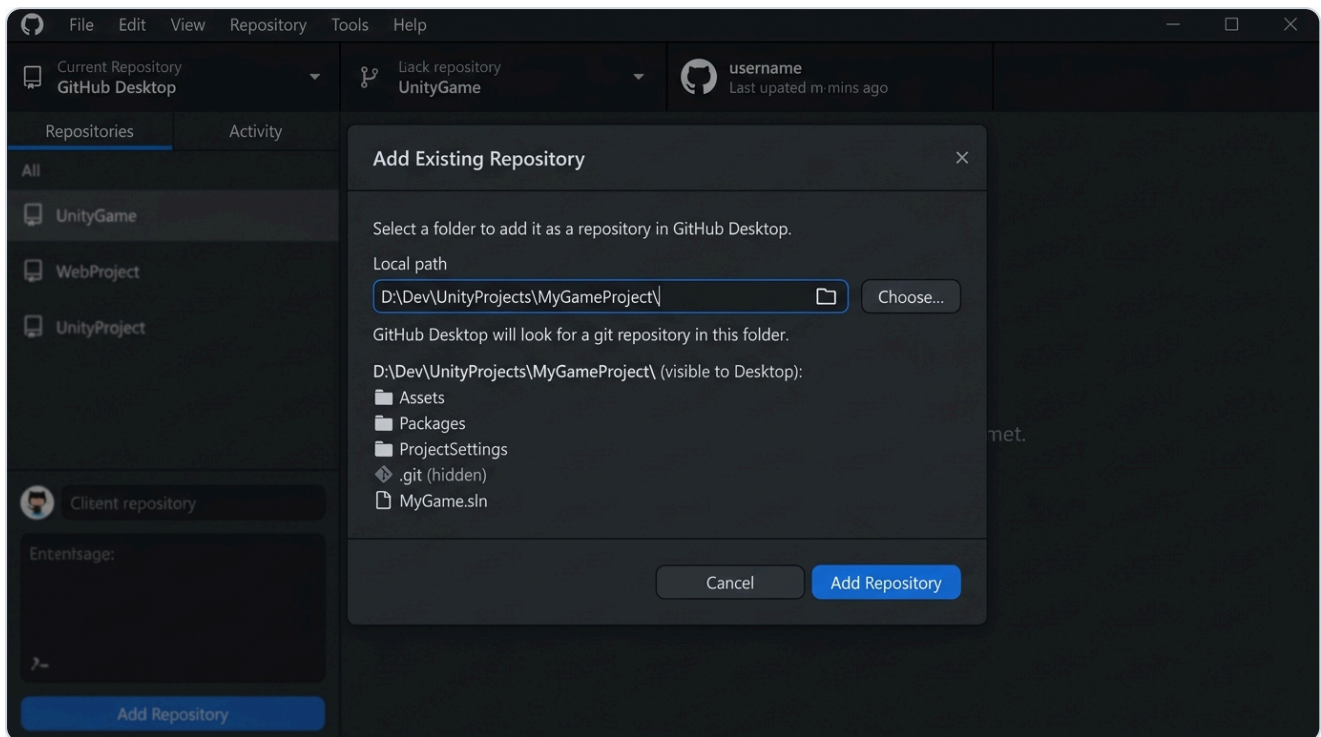


Figura 2.1: Cuadro de diálogo 'Add Existing Repository' en GitHub Desktop en macOS vinculando la carpeta del proyecto Unity.

[↑ Volver al Índice de Contenidos](#)

2.2 Clonación Correcta de Proyectos con Git LFS en macOS

Modalidad A: Vía Terminal Zsh

```
git lfs install
git clone git@github.com:mi-organizacion/videojuego.git
cd videojuego
git lfs pull
```

Modalidad B: Vía GitHub Desktop

1. Pulsa en **File** → **Clone Repository** (o `Cmd + Shift + O`).
2. Selecciona el repositorio de la lista o escribe la URL.
3. Elige la ruta de destino local en tu Mac y pulsa **Clone**.
4. GitHub Desktop gestionará la descarga completa de los assets pesados de LFS automáticamente.

[↑ Volver al Índice de Contenidos](#)

2.3 El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits

Modalidad A: Vía Terminal Zsh



```
# 1. Comprobar archivos modificados
git status

# 2. Agregar scripts y prefabs asegurando sus respectivos .meta
git add Assets/Scripts/PlayerController.cs Assets/Scripts/PlayerController.cs.meta
git add Assets/Prefabs/Player.prefab Assets/Prefabs/Player.prefab.meta

# 3. Confirmar commit
git commit -m "feat(player): implementar salto con fisica Rigidbody2D"
```

Modalidad B: Vía GitHub Desktop

1. En la pestaña **Changes** de la barra lateral izquierda, inspecciona los archivos modificados.
2. Comprueba que cada script o prefab tenga marcada su casilla junto a su archivo `.meta`.
3. En el panel derecho, verifica el diff de cambios con colores verde (añadido) y rojo (eliminado).
4. Introduce el título en **Summary** y contexto adicional en **Description**.
5. Pulsa en el botón azul **Commit to main**.

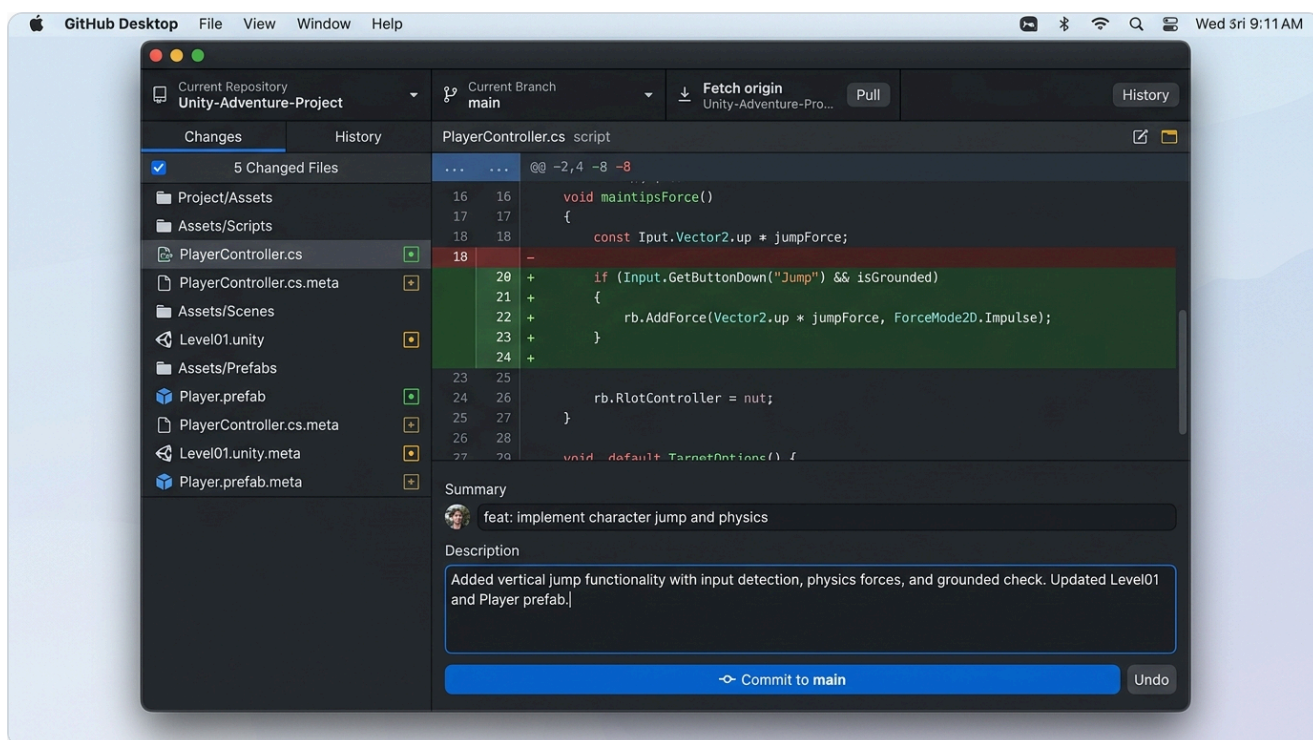


Figura 2.2: Pestaña de cambios en GitHub Desktop verificando en macOS la consistencia de archivos y metadatos de Unity.

[↑ Volver al Índice de Contenidos](#)

2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos

Prefijo	Área de Aplicación en Videojuegos	Ejemplo
feat:	Nuevas mecánicas o sistemas jugables	feat(player): anadir aceleracion y derrape en curvas
fix:	Corrección de fallos o glitches visuales	fix(metal): corregir artefacto de reflejo en shaders Metal
art:	Modelos 3D, texturas y animaciones	art(hero): importar animacion de correr y salto doble
level:	Modificaciones de escenas y diseño de niveles	level(volcano): anadir zonas de lava con dano continuo
audio:	Efectos de sonido y mezclas de AudioManager	audio(fx): balancear volumen de pasos en superficies metalicas
perf:	Optimizaciones de draw calls y memoria	perf(shaders): optimizar shaders URP para GPUs Apple Silicon
chore:	Mantenimiento de paquetes UPM o gitignore	chore(upm): actualizar Unity Collections a version 2.1.4

[↑ Volver al Índice de Contenidos](#)

2.5 Sincronización sin Romper la Cache (`pull --rebase` vs Fetch en Desktop)

Modalidad A: Vía Terminal Zsh

```
git pull --rebase origin main
git push origin main
```

El rebase mantiene la historia lineal, previniendo que Unity invalide cachés de compilación locales al recibir cambios.

Modalidad B: Vía GitHub Desktop

1. Pulsa en **Fetch origin** en la barra superior.
2. Si hay commits nuevos, pulsa en el botón **Pull origin**.

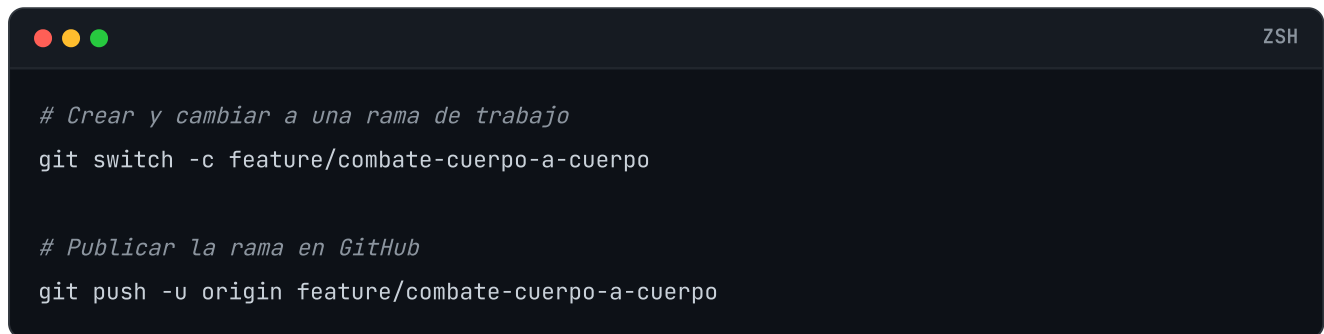
3. Envía tus cambios pendientes haciendo clic en **Push origin**.

[↑ Volver al Índice de Contenidos](#)

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Estrategia de Ramas en Equipos de Videojuegos (Zsh y Desktop)

Modalidad A: Vía Terminal Zsh



```
# Crear y cambiar a una rama de trabajo
git switch -c feature/combate-cuerpo-a-cuerpo

# Publicar la rama en GitHub
git push -u origin feature/combate-cuerpo-a-cuerpo
```

Modalidad B: Vía GitHub Desktop

1. En la barra superior, haz clic en el menú **Current Branch**.
2. Escribe el nombre de la nueva rama (ej. `feature/combate-cuerpo-a-cuerpo`).
3. Haz clic en el botón azul **New branch...**
4. Haz clic en **Publish branch** para sincronizarla con GitHub.

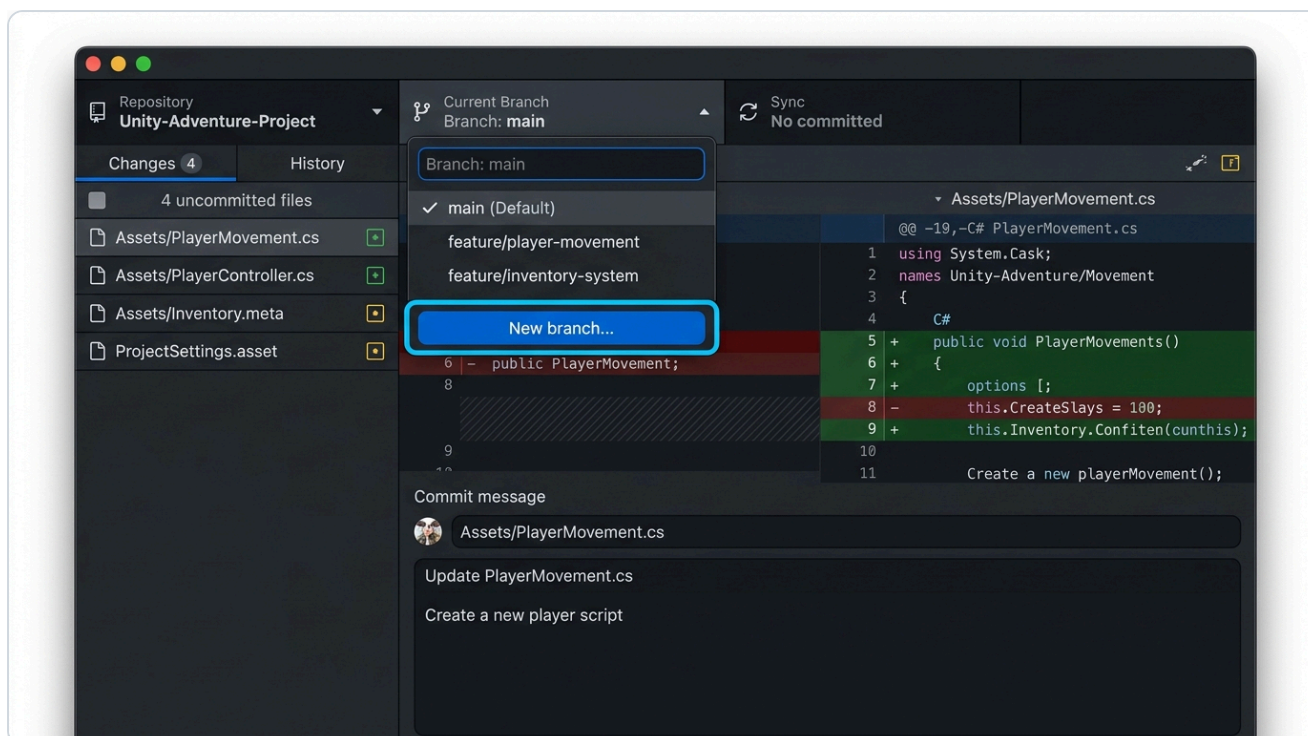


Figura 3.1: Menú desplegable 'Current Branch' en GitHub Desktop facilitando la creación y conmutación de ramas de trabajo en macOS.

[↑ Volver al Índice de Contenidos](#)

3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)

Para evitar que artistas y programadores choquen al editar la misma escena, subdivide el mundo en subescenas:

```

Nivel_Montaña/
├─ Nivel_Core.unity      → Cámaras, GameManagers, UI Canvas.
├─ Nivel_Entorno.unity   → Terreno, mallas y vegetación (Artistas).
├─ Nivel_Iluminacion.unity → Luces y Post-Processing (Iluminadores).
└─ Nivel_Gameplay.unity  → Enemigos, checkpoints y triggers (Diseñadores).
  
```

Script C# para Carga Aditiva en Tiempo de Ejecución:

Guarda en `Assets/Scripts/AdditiveLoader.cs` :

```

using UnityEngine;
using UnityEngine.SceneManagement;

public class AdditiveLoader : MonoBehaviour
{
    [SerializeField] private string[] additiveScenes = {
        "Nivel_Entorno",
        "Nivel_Iluminacion",
        "Nivel_Gameplay"
    };

    private void Start()
    {
        foreach (string scene in additiveScenes)
        {
            if (!SceneManager.GetSceneByName(scene).isLoaded)
            {
                SceneManager.LoadSceneAsync(scene, LoadSceneMode.Additive);
            }
        }
    }
}

```

[↑ Volver al Índice de Contenidos](#)

3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes

- Trabaja siempre dentro del **Prefab Mode** de Unity.
- Usa **Prefab Variants** para variantes de enemigos o props. Las variantes solo guardan las diferencias (*overrides*), evitando conflictos de fusión en el archivo base.

[↑ Volver al Índice de Contenidos](#)

3.4 Configuración de UnityYAMLMerge en macOS y Personalización de Fallback en `mergespecfile.txt`

En macOS, `UnityYAMLMerge` se encuentra ubicado dentro del bundle `.app` del editor en `/Applications/Unity/Hub/Editor/<version>/Unity.app/Contents/Tools/`.

Paso 1: Configurar el Driver en Git con Zsh

```
ZSH

# Detectar la versión más reciente del editor instalado en Applications
UNITY_VERSION=$(ls -1 /Applications/Unity/Hub/Editor 2>/dev/null | tail -n 1)
MERGE_TOOL="/Applications/Unity/Hub/Editor/${UNITY_VERSION}/Unity.app/Contents/Tools/UnityYAMLMerge"

if [ -f "$MERGE_TOOL" ]; then
    git config --global merge.unityyamlmerge.name "Unity Smart Merge"
    git config --global merge.unityyamlmerge.driver "\"${MERGE_TOOL}\" merge -p -- '%0' '%B' '%A' '%A'"
    git config --global merge.unityyamlmerge.trustExitCode true
    git config --global merge.unityyamlmerge.recursive binary
    echo "✓ UnityYAMLMerge configurado correctamente en: $MERGE_TOOL"
else
    echo "⚠ Advertencia: No se encontró Unity Editor en /Applications/Unity/Hub/Editor/"
fi
```

Paso 2: Personalización del Fallback en `mergespecfile.txt`

En la misma carpeta del ejecutable (/Applications/Unity/Hub/Editor/<version>/Unity.app/Contents/Tools/) se encuentra mergespecfile.txt. Configúralo para invocar tu editor favorito cuando existan colisiones en los mismos campos YAML:

```
INI

# Configuración recomendada para macOS en mergespecfile.txt

# Opción A: Visual Studio Code
* use "code" --wait --merge "%b" "%t" "%d" "%d"

# Opción B: JetBrains Rider
* use "/Applications/Rider.app/Contents/MacOS/rider" merge "%b" "%t" "%d" "%d"

# Opción C: Xcode FileMerge (opendiff)
* use "opendiff" "%b" "%t" -ancestor "%d" -merge "%d"

# Opción D: Beyond Compare para Mac
* use "/Applications/Beyond Compare.app/Contents/MacOS/bcomp" "%b" "%t" "%d" "%d"
```

NOTE

Con esta configuración, Git ejecuta primero `UnityYAMLMerge` para unir automáticamente los elementos estructurales no conflictivos. Si dos ramas cambiaron concurrentemente una misma variable del inspector, se abre de inmediato la interfaz visual para su resolución asistida.

[↑ Volver al Índice de Contenidos](#)

3.5 Uso de Git Stash, Historial y Reversión Segura (Zsh y Desktop)

Modalidad A: Vía Terminal Zsh

```

# Guardar cambios sin confirmar
git stash save "WIP: ajuste de shaders Metal"

# Cambiar de rama y revisar bug
git switch main

# Restaurar cambios
git switch feature/mi-rama
git stash pop
```

Modalidad B: Vía GitHub Desktop

1. Al conmutar de rama con cambios sin confirmar, pulsa en "**Leave my changes on [rama-actual]**" (crea un stash).
2. Para restaurar, pulsa en **Restore** dentro de **Stashed Changes** en la barra lateral.
3. Para deshacer un commit defectuoso: ve a la pestaña **History**, haz clic derecho sobre el commit → **Revert changes in commit**.

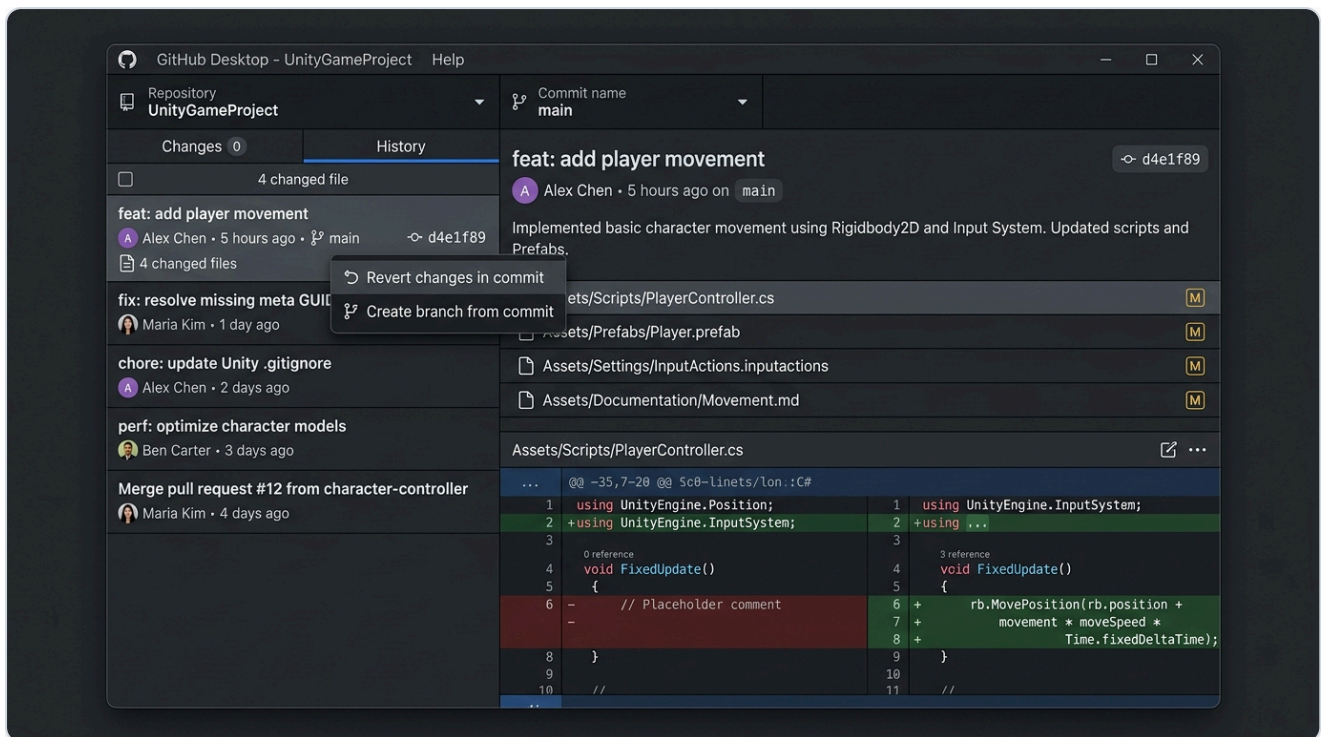


Figura 3.2: Pestaña 'History' en GitHub Desktop en macOS permitiendo inspección de diffs y reversión limpia de commits problemáticos.

[↑ Volver al Índice de Contenidos](#)

3.6 Modularización de Código con Assembly Definitions (`.asmdef` y `.asmref`)

En macOS con chips Apple Silicon, la compilación de scripts en Unity es extremadamente rápida gracias al ancho de banda de la memoria unificada. Sin embargo, compilar todo el proyecto en una única DLL monolítica (`Assembly-CSharp.dll`) deteriora los tiempos de respuesta en cada guardado.

La Solución Profesional: Archivos `.asmdef``

Divide tu código fuente en sub-ensamblados modulares:

- `Assets/_Project/Core/Game.Core.asmdef`
- `Assets/_Project/Gameplay/Game.Gameplay.asmdef` (con referencia a `Game.Core`)
- `Assets/_Project/UI/Game.UI.asmdef` (con referencia a `Game.Core`)
- `Assets/_Project/Editor/Game.Editor.asmdef` (restringido a plataforma Editor)

```
{
  "name": "Game.Gameplay",
  "rootNamespace": "Game.Gameplay",
  "references": [
    "Game.Core",
    "Unity.InputSystem"
  ],
  "includePlatforms": [],
  "excludePlatforms": [],
  "allowUnsafeCode": false,
  "overrideReferences": false,
  "precompiledReferences": [],
  "autoReferenced": true,
  "defineConstraints": [],
  "versionDefines": [],
  "noEngineReferences": false
}
```

Beneficios en macOS y Git:

1. **Compilación Instantánea:** Al editar un script en Rider o VS Code, Unity recompila únicamente el módulo modificado en menos de 1 segundo.
2. **Pull Requests Claros:** Los revisores distinguen en GitHub qué submódulo del videojuego ha sido intervenido.
3. **Arquitectura Limpia:** Elimina acoplamientos y dependencias no deseadas entre sistemas de juego.

[↑ Volver al Índice de Contenidos](#)

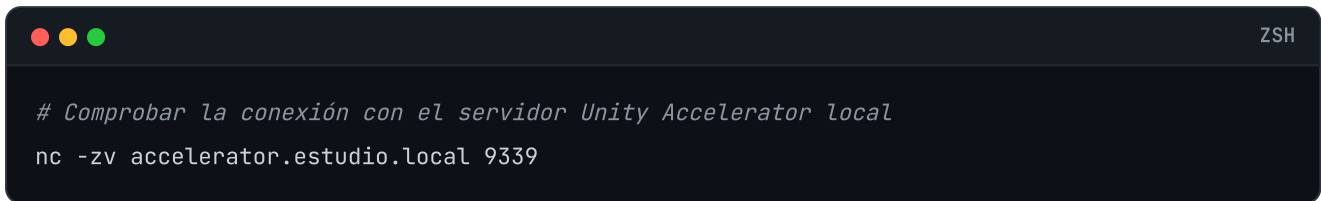
3.7 Unity Accelerator: Aceleración de Descargas y Caché de Importación en LAN

Cuando varios desarrolladores en macOS clonan o hacen `git pull` de assets pesados (modelos 3D y texturas de alta resolución), el Editor local debe transcodificar cada textura a compresión ASTC para Apple Silicon.

Unity Accelerator almacena en caché en la red local (LAN) los assets ya procesados:

1. El primer equipo que importa un asset sube los metadatos y binarios transcodificados al Accelerator por el puerto 9339.

2. Los demás desarrolladores al hacer `git pull` descargan el asset precompilado por la red local a través de Gigabit/10Gbe o Wi-Fi 6E, evitando la sobrecarga térmica y reduciendo tiempos de 30 minutos a escasos segundos.



```
# Comprobar la conexión con el servidor Unity Accelerator local
nc -zv accelerator.estudio.local 9339
```

[↑ Volver al Índice de Contenidos](#)

Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente

4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity

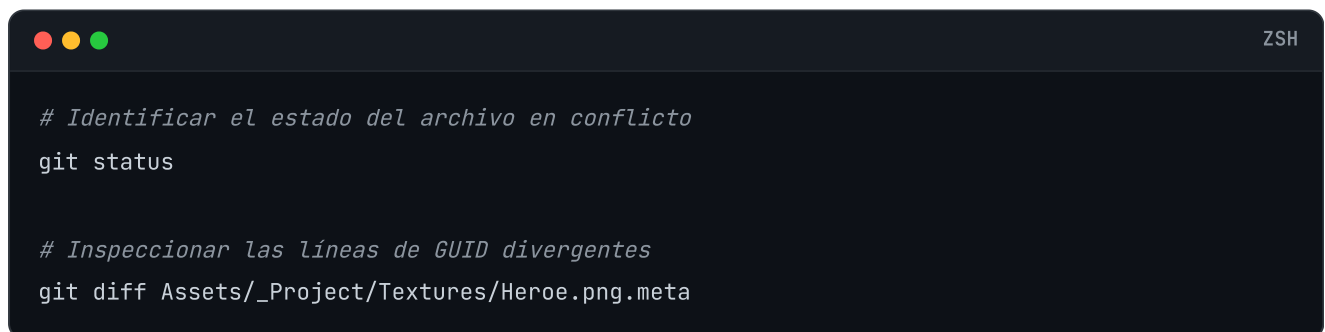
Los conflictos no se resuelven en la terminal; se evitan desde la arquitectura del proyecto:

1. **Multi-Scene Editing Aditivo:** Distribuye las responsabilidades en subescenas funcionales (Geometría, Luces, Jugabilidad).
2. **Uso Exclusivo de Prefabs:** Ningún GameObject dinámico debe residir huérfano en la jerarquía de la escena.
3. **Locks de LFS Activos:** Exige el bloqueo preventivo de texturas maestras de Photoshop (`.psd`) y modelos 3D (`.blend`) antes de editarlos.

[↑ Volver al Índice de Contenidos](#)

4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)

Se produce cuando dos desarrolladores añaden un asset con el mismo nombre o mueven carpetas simultáneamente, generando dos GUIDs dispares para el mismo recurso:



```
# Identificar el estado del archivo en conflicto
git status

# Inspeccionar las líneas de GUID divergentes
git diff Assets/_Project/Textures/Heroe.png.meta
```

Solución Paso a Paso:

1. Si el recurso ya fue referenciado en escenas o prefabs por tu compañero en el servidor remoto, **conserva el GUID remoto**.
2. Abre el `.meta` en VS Code o JetBrains Rider y elimina los marcadores de conflicto dejando una sola línea `guid: .`
3. Confirma la resolución:

```
git add Assets/_Project/Textures/Heroe.png.meta
git commit -m "fix(meta): resolver colision de GUID en textura Heroe"
```

[↑ Volver al Índice de Contenidos](#)

4.3 Tema 3: Conflictos en Scripts C# (`.cs`)

Ocurre cuando dos programadores editan la misma sección o método dentro de un script C#:

```
<<<<<< HEAD
    void Jump() {
        rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
    }
=====
    void Jump() {
        if (isGrounded) {
            rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpSpeed);
        }
    }
>>>>>> feature/salto-mejorado
```

Solución:

1. Abre el script en VS Code o JetBrains Rider y unifica ambas intenciones:

```
void Jump() {
    if (isGrounded) {
        rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
    }
}
```

2. Guarda el archivo y confírmalo en Zsh:

```
git add Assets/_Project/Scripts/PlayerController.cs
git commit -m "fix(player): unificar validacion de suelo con impulso de salto"
```

[↑ Volver al Índice de Contenidos](#)

4.4 Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge (Zsh y Desktop)

Cuando dos desarrolladores agregan o modifican GameObjects diferentes en la misma escena:

Modalidad A: Vía Terminal Zsh con UnityYAMLMerge

```
git mergetool -t unityyamlmerge
```

¿Qué hace este comando?

Invoca el binario `UnityYAMLMerge` de macOS, reconstruyendo el árbol semántico y uniendo los GameObjects agregados por ambas partes sin corromper los identificadores internos `fileID`.

Modalidad B: Vía GitHub Desktop

1. Tras un merge o rebase conflictivo, aparece el cuadro modal **Resolve conflicts before merging**.
2. En la lista de archivos con advertencia, pulsa en el menú desplegable junto a la escena `.unity`.
3. Selecciona **Open in UnityYAMLMerge** (o tu herramienta configurada en `mergespecfile.txt`).
4. Al completarse la fusión tridireccional, haz clic en el botón azul **Resolve Conflicts**.

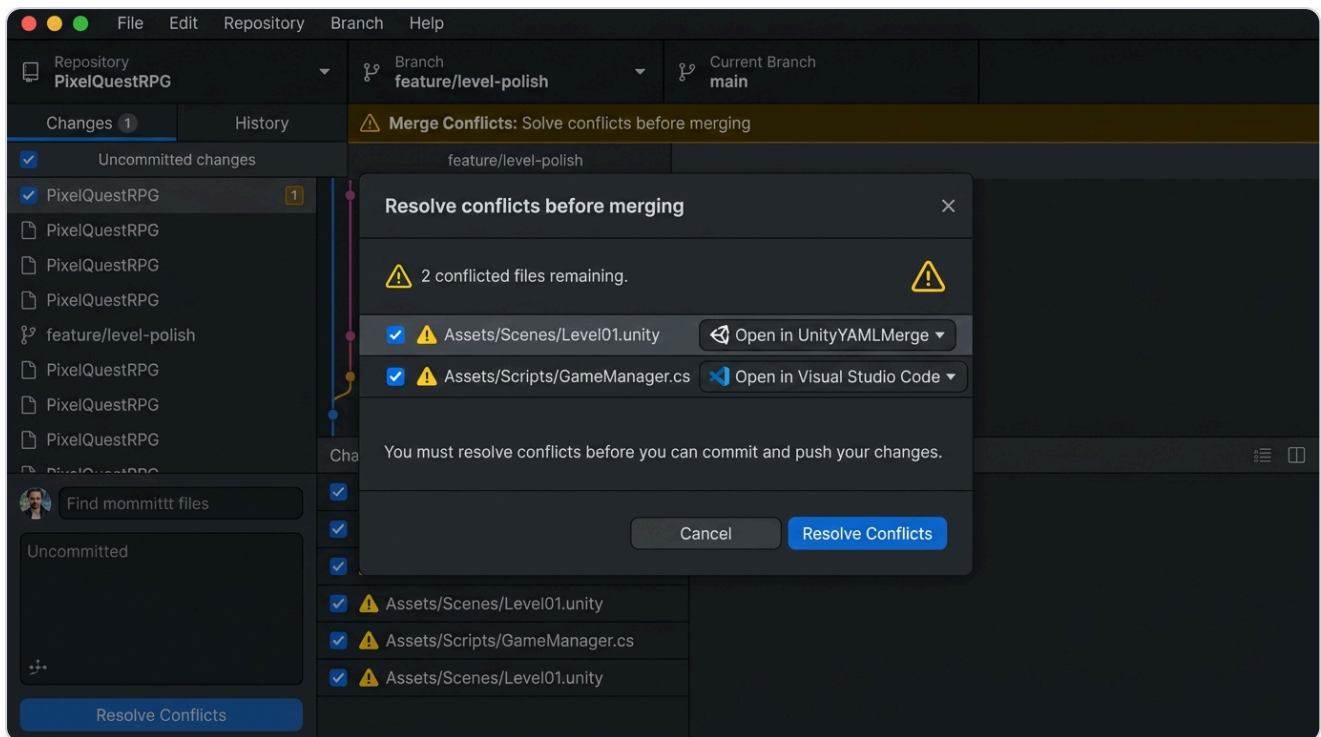


Figura 4.1: Ventana de resolución de conflictos en GitHub Desktop permitiendo derivar la escena a UnityYAMLMerge o elegir versiones completas.

[↑ Volver al Índice de Contenidos](#)

4.5 Tema 5: Forzar una Versión Completa de Asset (`--ours` vs `--theirs`)

Si una escena o prefab se corrompió irreversiblemente durante una fusión y el equipo decide descartar por completo una de las dos versiones:

Modalidad A: Vía Terminal Zsh

```

# Opción 1: Conservar tu versión local intacta y descartar la del servidor
git checkout --ours Assets/_Project/Scenes/Nivel01.unity
git add Assets/_Project/Scenes/Nivel01.unity
git commit -m "resolve: conservar version local de Nivel01"

# Opción 2: Descartar tu trabajo local y aceptar íntegramente la versión remota
git checkout --theirs Assets/_Project/Scenes/Nivel01.unity
git add Assets/_Project/Scenes/Nivel01.unity
git commit -m "resolve: adoptar version remota de Nivel01"

```

Modalidad B: Vía GitHub Desktop

En la ventana modal de conflictos, haz clic en la flecha derecha del archivo y selecciona **Use Modified Version** (tuya) o **Use Existing Version** (remota).

[↑ Volver al Índice de Contenidos](#)

4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull

Si el Editor de Unity guardó automáticamente las escenas al presionar el botón Play y tienes modificaciones sucias que impiden realizar `git pull`:

```
git stash save "guardado-automatico-editor"
git pull --rebase origin main
git stash pop
```

Si al recuperar el stash surgen colisiones en archivos de cache local, descarta los temporales con:

```
git checkout -- Assets/_Project/Scenes/AutoSave.unity
```

[↑ Volver al Índice de Contenidos](#)

4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS

Ocurre cuando otro desarrollador subió commits a GitHub mientras tú trabajabas localmente:

```
# 1. Obtener los metadatos y punteros LFS remotos
git fetch origin

# 2. Reorganizar tus commits locales por encima de los remotos
git rebase origin/main

# 3. Subir de forma lineal y limpia
git push origin main
```

El uso estricto de `rebase` mantiene una línea temporal limpia y garantiza que los objetos de Git LFS se descarguen secuencialmente sin merge commits redundantes.

[↑ Volver al Índice de Contenidos](#)

4.8 Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock

Los archivos binarios (`.fbx` , `.psd` , `.blend` , `.wav`) no admiten fusión por diferencias de líneas. Para impedir que dos artistas trabajen sobre el mismo archivo a la vez en macOS:

```

# 1. Bloquear el archivo en el servidor antes de comenzar a pintar o modelar
git lfs lock Assets/_Project/Art/Personajes/Heroe.psd

# 2. Verificar qué archivos están bloqueados en el repositorio y por qué usuario
git lfs locks

# 3. Trabajar en Photoshop o Blender, guardar, commitear y subir
git add Assets/_Project/Art/Personajes/Heroe.psd
git commit -m "art(heroe): texturizar mascara y detalles de armadura"
git push origin main

# 4. Desbloquear para que el resto del equipo pueda editarlo
git lfs unlock Assets/_Project/Art/Personajes/Heroe.psd
```

[↑ Volver al Índice de Contenidos](#)

4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano

Si un desarrollador elimina un archivo desde el Finder de macOS pero olvida su `.meta` , Git registrará un conflicto de metadato huérfano:

```

# Eliminar metadato huérfano del control de versiones
git rm Assets/_Project/Scripts/OldManager.cs.meta
git commit -m "chore: purgar archivo .meta huerfano tras remocion de script"
```

[↑ Volver al Índice de Contenidos](#)

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees en macOS: Múltiples Ramas sin Recargar `Library/`

Conmutar de rama en un proyecto de 50 GB obliga al editor a reimportar la carpeta `Library/` durante 20 minutos. Con **Git Worktrees** puedes tener dos ramas abiertas en carpetas independientes del disco en macOS:

```

# Crear worktree paralelo en otra carpeta del disco
git worktree add ../MiJuego-Hotfix hotfix/correccion-audio

# Trabajar en la carpeta secundaria sin tocar el Unity principal
cd ../MiJuego-Hotfix
git push origin hotfix/correccion-audio

# Al terminar, eliminar el worktree de forma limpia
cd ../MiJuegoMac
git worktree remove ../MiJuego-Hotfix
```

Beneficio clave:

Tu instancia principal de Unity no sufre ninguna reimportación de assets ni pierde la caché de compilación de shaders Metal de Apple Silicon.

[↑ Volver al Índice de Contenidos](#)

5.2 Depuración Binaria con Git Bisect y Blame en C#

Cuando una mecánica falla y se desconoce qué commit introdujo la regresión:

```

# Iniciar la búsqueda binaria
git bisect start
git bisect bad           # El commit actual contiene el fallo
git bisect good v1.0.4    # La versión v1.0.4 funcionaba correctamente

# Git cambiará automáticamente de commit intermedio.
# Prueba en Unity y clasifica:
git bisect good # o: git bisect bad

# Al culminar, Git te señalará el commit exacto causante del bug.
git bisect reset

```

[↑ Volver al Índice de Contenidos](#)

5.3 GitHub Codespaces desde la Terminal de Mac

Para editar scripts C#, shaders Metal o archivos de configuración sin necesidad de encender la estación de trabajo completa:

```

gh codespace create --repo mi-organizacion/MiVideojuegoMac --branch main

```

Permite revisar PRs, editar código C# con intellisense y compilar librerías en la nube con VS Code en el navegador.

[↑ Volver al Índice de Contenidos](#)

5.4 GitHub Copilot CLI en Zsh

```

# Consultar comandos complejos de Git para Unity en macOS
gh copilot suggest "como revertir un commit de una escena de unity sin tocar scripts"
gh copilot explain "git lfs push --all origin main"

```

[↑ Volver al Índice de Contenidos](#)

5.5 Git Hooks y Validación Pre-commit de Archivos `.meta` en macOS

Crea el archivo `.git/hooks/pre-commit` para evitar que ningún miembro del equipo suba un asset sin su archivo `.meta`:

```
BASH

#!/bin/sh
# Pre-commit hook para macOS: Validar integridad de archivos .meta
MISSING=0
for f in $(git diff --cached --name-only --diff-filter=A | grep '^Assets/'); do
    if [ "${f##*.}" != "meta" ]; then
        if [ ! -f "${f}.meta" ] && ! git diff --cached --name-only | grep -q "^${f}.meta$";
    then
        echo "❌ ERROR: Falta el archivo .meta para: $f"
        MISSING=1
    fi
fi
done

if [ $MISSING -eq 1 ]; then
    echo "🚫 Commit rechazado: Todo asset de Unity debe incluir su archivo .meta correspondiente."
    exit 1
fi
exit 0
```

```
ZSH

chmod +x .git/hooks/pre-commit
```

[↑ Volver al Índice de Contenidos](#)

5.6 Descargas Parciales y Ahorro de Cuota de Git LFS con `lfs.fetchexclude` y `git sparse-checkout`

En proyectos de escala AAA donde el repositorio de Unity supera los 100 GB, los ingenieros de gameplay o programadores de gráficos en Mac no necesitan clonar la totalidad de archivos de audio sin compresión ni cinemáticas 4K.

Filtrado de Descarga con `lfs.fetchexclude` en Zsh

```
# Excluir de la descarga de LFS carpetas de assets de niveles posteriores
git config lfs.fetchexclude "Assets/_Project/Art/Levels/Nivel04/*,
Assets/_Project/Cinematics/*"

# Descargar solo los archivos requeridos localmente
git lfs pull
```

Los assets excluidos quedarán en disco como ligeros punteros de texto (130 bytes). Unity compilará el proyecto sin error y tu disco NVMe ahorrará decenas de gigabytes.

Sparse-Checkout para Repositorios Gigantes

Clona únicamente las carpetas que necesitas para trabajar:

```
# Inicializar sparse checkout en modo cono
git sparse-checkout init --cone

# Seleccionar los directorios funcionales del proyecto
git sparse-checkout set Assets/_Project/Scripts Assets/_Project/Core Packages
ProjectSettings
```

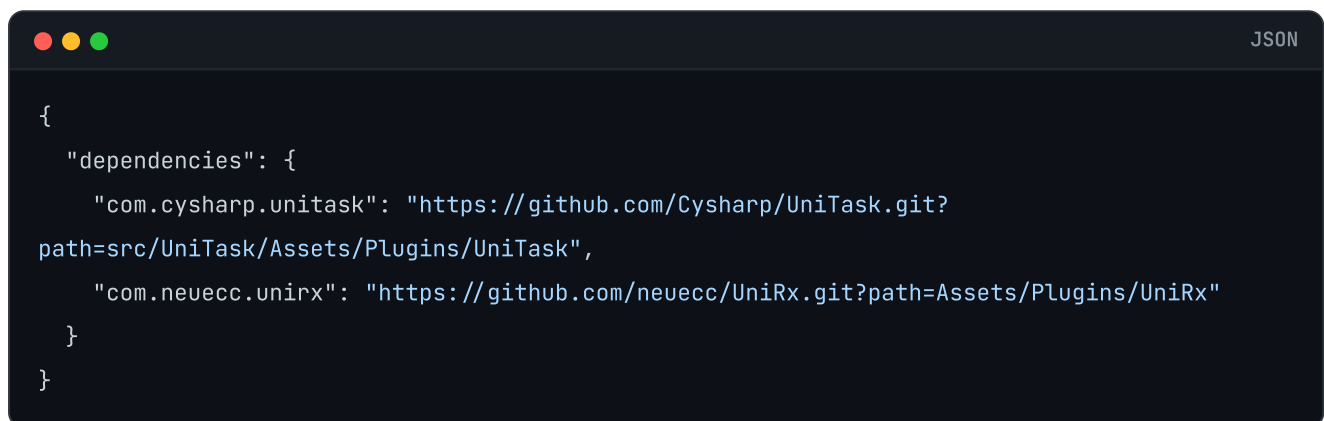
[↑ Volver al Índice de Contenidos](#)

Parte VI: Gestión de Paquetes UPM y Dependencias

6.1 Instalación de Paquetes Mediante URLs de Git en UPM

Unity Package Manager permite consumir paquetes directamente desde repositorios de GitHub.

Añade a `Packages/manifest.json` :

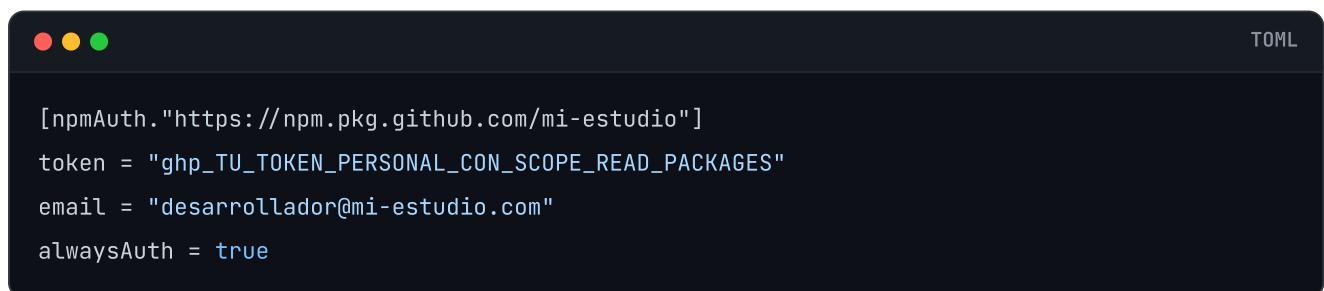


```
{
  "dependencies": {
    "com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?path=src/UniTask/Assets/Plugins/UniTask",
    "com.nevecc.unirx": "https://github.com/nevecc/UniRx.git?path=Assets/Plugins/UniRx"
  }
}
```

[↑ Volver al Índice de Contenidos](#)

6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens

Configura en `~/.upmconfig.toml` :



```
[npmAuth."https://npm.pkg.github.com/mi-estudio"]
token = "ghp_TU_TOKEN_PERSONAL_CON_SCOPE_READ_PACKAGES"
email = "desarrollador@mi-estudio.com"
alwaysAuth = true
```

[↑ Volver al Índice de Contenidos](#)

6.3 Addressables Asset System frente a `Resources/`: Versionado y Despliegue en CDN

La utilización de `Resources/` en macOS infla drásticamente el tamaño del paquete `.app` final y destruye el streaming asíncrono de texturas en la GPU Metal de Apple Silicon.

La Solución: Addressables (`com.unity.addressables`)

Addressables desacopla las dependencias mediante `AssetReference`, permitiendo empaquetar AssetBundles y alojarlos en CDNs remotas (Cloudflare R2, AWS S3).

Qué se versiona en Git y qué se ignora:

- **En Git:** Versiona `Assets/AddressableAssetSettings/` (catálogos YAML, perfiles de entorno y esquemas de grupos).
- **Fuera de Git:** Ignora `ServerData/` (carpeta local donde Unity compila los AssetBundles binarios).

PLAINTEXT

```
# Exclusión de compilación local de Addressables en .gitignore
[ServerData/
```

[↑ Volver al Índice de Contenidos](#)

Parte VII: Automatización CI/CD con GitHub Actions y GameCI

7.1 Arquitectura de GameCI para macOS Standalone (Apple Silicon)

Compila automáticamente aplicaciones de macOS (`.app` y `.dmg`) optimizadas para Apple Silicon en runners de GitHub Actions.

[↑ Volver al Índice de Contenidos](#)

7.2 Activación de Licencias de Unity en GitHub Actions

En **Settings** → **Secrets and variables** → **Actions**, define los siguientes secretos:

- `UNITY_EMAIL` : Tu correo de la cuenta de Unity.
- `UNITY_PASSWORD` : Tu contraseña de Unity.
- `UNITY_LICENSE` : El contenido en texto de tu archivo de licencia (`.ulf`).

[↑ Volver al Índice de Contenidos](#)

7.3 Pipeline Automatizado: Tests y Compilación StandaloneOSX (.app / .dmg)

Crea `.github/workflows/build-macos.yml` :

```

name: Unity macOS Build

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    name: 🖋 Pruebas Unitarias EditMode y PlayMode
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v3
        with:
          path: Library
          key: Library-test-${{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}

      - uses: game-ci/unity-test-runner@v4
        env:
          UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
          UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
          UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
        with:
          githubToken: ${ secrets.GITHUB_TOKEN }

  build:
    name: 🍏 Compiler StandaloneOSX Universal
    needs: test
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v3

```

```

    with:
      path: Library
      key: Library-macOS-${{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}
  }}

- uses: game-ci/unity-builder@v4
  env:
    UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
    UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
    UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
  with:
    targetPlatform: StandaloneOSX
    buildName: MiJuegoMac

- uses: actions/upload-artifact@v4
  with:
    name: Build-macOS-Universal
    path: build/StandaloneOSX

```

[↑ Volver al Índice de Contenidos](#)

7.4 Subida Automática de Builds a GitHub Releases

El pipeline genera un archivo `.zip` o `.dmg` descargable con la aplicación nativa lista para su ejecución en macOS.

[↑ Volver al Índice de Contenidos](#)

7.5 Pruebas Automatizadas con Code Coverage y Reportes en GitHub Actions

Valida en cada Pull Request la cobertura de pruebas de código mediante `com.unity.test-framework.code-coverage` :

```
- name: Ejecutar Tests con Cobertura de Código
  uses: game-ci/unity-test-runner@v4
  env:
    UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
    UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
    UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
  with:
    githubToken: ${ secrets.GITHUB_TOKEN }
    customParameters: -enableCodeCoverage -coverageResultsPath ./coverage-results -
coverageOptions generateAdditionalMetrics;generateHtmlReport;generateBadgeReport

- name: Publicar Reporte de Cobertura
  uses: actions/upload-artifact@v4
  with:
    name: Code-Coverage-Report
    path: ./coverage-results
```

[↑ Volver al Índice de Contenidos](#)

Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos

8.1 Branch Protection Rules y Rulesets

Protege la rama `main` en GitHub:

1. Obliga a pasar los tests automáticos de GameCI antes de fusionar.
2. Bloquea pushes directos forzando el uso de Pull Requests con al menos una aprobación obligatoria de un revisor técnico.
3. Prohíbe terminantemente `git push --force`.

[↑ Volver al Índice de Contenidos](#)

8.2 Gestión de Secretos para APIs de Juegos (Apple Game Center, Steam)

Guarda las credenciales en variables de entorno o archivos `.env` ignorados por Git. Nunca committees certificados de desarrollo de Apple o claves de Game Center en scripts C#.

[↑ Volver al Índice de Contenidos](#)

8.3 Gobernanza con `CODEOWNERS` para Artistas y Programadores

Crea `.github/CODEOWNERS` :

```
Assets/Scripts/      @lead-programmer
Assets/Art/           @lead-artist
Assets/Scenes/        @level-design-lead
ProjectSettings/      @tech-director
```

[↑ Volver al Índice de Contenidos](#)

Parte IX: Distribución y Despliegue con GitHub Releases

9.1 Creación Automatizada de Releases con Tags Semánticos

```
git tag -a v1.0.0 -m "release: version inicial 1.0.0 para macOS"
git push origin v1.0.0
```

[↑ Volver al Índice de Contenidos](#)

9.2 Empaquetado y Publicación de Archivos DMG y Aplicaciones macOS

```
# Crear archivo comprimido de la aplicación macOS
tar -czf MiJuego-v1.0.0-macOS.tar.gz -C build/StandaloneOSX MiJuegoMac.app

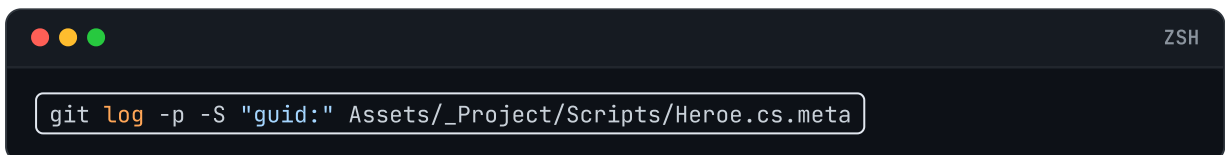
# Publicar el release con GitHub CLI
gh release create v1.0.0 MiJuego-v1.0.0-macOS.tar.gz \
  --title "Mi Videojuego v1.0.0 (macOS Apple Silicon)" \
  --notes "Compilación oficial optimizada para chips Apple Silicon (M1/M2/M3/M4) con backend gráfico Metal."
```

[↑ Volver al Índice de Contenidos](#)

Parte X: Catálogo Maestro de Incidentes Críticos de Unity en macOS

10.1 Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs

- **Síntoma:** Todos los componentes de los personajes o enemigos muestran en el Inspector: `The associated script can not be loaded. Please fix any compile errors...`.
- **Causa:** Un desarrollador eliminó y recreó el archivo `.meta` de un script, asignándole un nuevo GUID aleatorio que rompió todas las referencias previas de las escenas.
- **Solución de Rescate:**
 1. Busca el GUID original en el historial de Git:



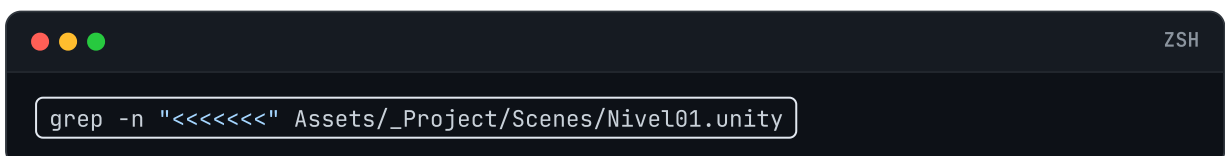
```
git log -p -S "guid:" Assets/_Project/Scripts/Heroe.cs.meta
```

2. Edita `Assets/_Project/Scripts/Heroe.cs.meta` y restaura el valor `guid:` original.
3. Abre Unity y recarga los assets con **Assets → Reimport All**.

[↑ Volver al Índice de Contenidos](#)

10.2 Incidente 2: Escena Corrupta por Edición Manual o Conflicto Mal Resuelto

- **Síntoma:** Al abrir la escena, Unity arroja el error: `Scene 'Nivel01.unity' is damaged and could not be opened`.
- **Solución:**
 1. Busca marcadores residuales de conflicto:



```
grep -n "<<<<<<" Assets/_Project/Scenes/Nivel01.unity
```

2. Elimina todas las líneas de marcadores de conflicto dejando el archivo YAML válido.

3. Si la escena continúa corrupta, descarta los cambios locales volviendo a la versión del último commit funcional:

```
git checkout HEAD -- Assets/_Project/Scenes/Nivel01.unity
```

[↑ Volver al Índice de Contenidos](#)

10.3 Incidente 3: Subida Accidental de la Carpeta `Library/` o Archivos `.DS\_Store`

- **Síntoma:** El comando `git push` sube gigabytes innecesarios y aparecen archivos ocultos `.DS_Store` en GitHub.
- **Solución:**

```
git rm -r --cached Library/  
find . -name ".DS_Store" -exec git rm --cached {} +  
git commit -m "fix(git): remover Library y archivos .DS_Store del repositorio"  
git push origin main
```

[↑ Volver al Índice de Contenidos](#)

10.4 Incidente 4: Repositorio Bloqueado por Superar el Límite de 100 MB

- **Síntoma:** `remote: error: File Assets/Models/Boss.fbx is 245.00 MB; this exceeds GitHub's file size limit of 100.00 MB .`
- **Solución de Rescate con `git-filter-repo`:**

```
brew install git-filter-repo  
git lfs migrate import --include="*.fbx,*.psd,*.blend,*.wav" --everything  
git push origin --force --all
```

10.5 Incidente 5: Shaders Magenta / Rosados tras Clonar en macOS

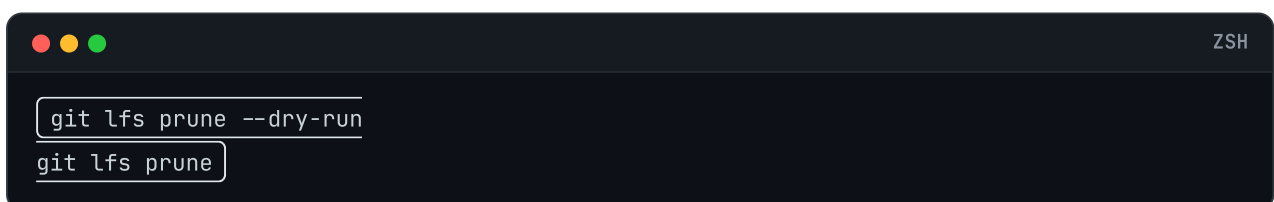
- **Síntoma:** Todos los materiales de la escena se ven de color rosa/magenta en el Editor de macOS.
- **Causa:** Incompatibilidad de pipeline gráfico o falta de compilación de shaders para Metal.
- **Solución:**
 1. Abre **Window** → **Package Manager** y verifica la instalación de `Universal RP`.
 2. Ejecuta en el menú: **Edit** → **Rendering** → **Materials** → **Convert Selected Built-in Materials to URP**.
 3. Purga la caché local de shaders si persiste:



```
rm -rf Library/ShaderCache
```

10.6 Incidente 6: Límite de Almacenamiento y Ancho de Banda de Git LFS Superado

- **Síntoma:** `Git LFS: Repository or organization has exceeded its bandwidth or storage quota`.
- **Solución:**



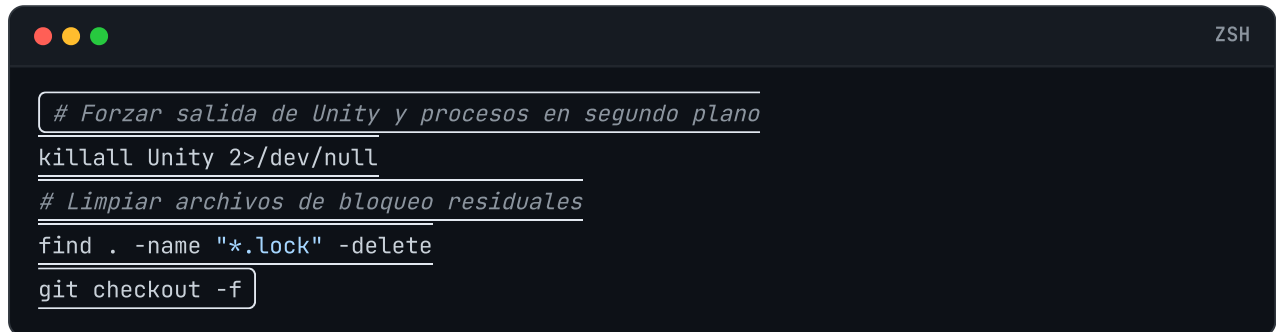
```
git lfs prune --dry-run
git lfs prune
```

10.7 Incidente 7: Desfase de Versiones Menores del Editor de Unity

- **Síntoma:** Escenas que cambian constantemente de formato en cada commit de Git.
- **Solución:** Abre siempre el proyecto a través de **Unity Hub**, el cual descargará e iniciará la versión exacta fijada en `ProjectSettings/ProjectVersion.txt`.

10.8 Incidente 8: Archivos Bloqueados por Procesos de Unity en Ejecución

- **Síntoma:** `git checkout` o `git merge` falla con errores de permisos o `cannot unlink`.
- **Solución:**



```
# Forzar salida de Unity y procesos en segundo plano
killall Unity 2>/dev/null
# Limpiar archivos de bloqueo residuales
find . -name "*.lock" -delete
git checkout -f
```

10.9 Incidente 9: Fuga de Claves Privadas en ScriptableObjects o Certificados Apple

- **Síntoma:** Se subió accidentalmente una clave de Game Center, Steam o un archivo `.p12`.
- **Solución:** Revoca los certificados o API keys inmediatamente y purga el archivo del historial con `git filter-repo`.

10.10 Incidente 10: Regeneración Limpia y Segura de la Caché Local en macOS

- **Síntoma:** Errores extraños de compilación en C#, referencias rotas que no desaparecen o fallos de renderizado que sólo le ocurren a un miembro del equipo en Mac.
- **Solución (El Reset Nuclear Seguro):**



A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner says "ZSH". The terminal contains three sections of text, each preceded by a horizontal line and a comment starting with "#". The first section shows the command "killall Unity 2>/dev/null". The second section shows the command "rm -rf Library/ Temp/ Obj/ Logs/ UserSettings/". The third section is a comment about reopening Unity Hub. The text is in a monospaced font.

```
# 1. Cerrar Unity
killall Unity 2>/dev/null

# 2. Eliminar directorios volátiles locales
rm -rf Library/ Temp/ Obj/ Logs/ UserSettings/

# 3. Reabrir desde Unity Hub para reconstruir Library de forma limpia y consistente
```

[↑ Volver al Índice de Contenidos](#)